

Guia rapido - comandos do sistema e-commerce

Arquivo principal: `main.cpp`

1. Bibliotecas

- `#include <iostream>` : entrada e saida no console, como `cin` e `cout`.
- `#include <fstream>` : leitura e gravacao de arquivos.
- `#include <vector>` : uso de lista dinamica com `vector`.
- `#include <cstring>` : funcoes para texto em `char[]`, como `strncpy`.
- `#include <algorithm>` : algoritmos prontos, como `sort`, `transform`, `max` e `swap`.
- `#include <cctype>` : funcoes de caractere, como `tolower`.
- `#include <iomanip>` : formatacao de tabela e casas decimais.
- `#include <limits>` : limites de tipos, usado para limpar o buffer.
- `#include <chrono>` : medicao de tempo das buscas e ordenacoes.
- `#include <string>` : uso do tipo `string`.
- `using namespace std` : evita escrever `std::` antes de comandos padrao.

2. Estruturas

- `struct Produto` : modelo dos dados de cada produto.
- `int codigo` : codigo numerico do produto.
- `char nome[60]` : nome do produto com limite de caracteres.
- `char categoria[40]` : categoria do produto.
- `double preco` : preco com casas decimais.
- `int estoque` : quantidade disponivel.
- `double avaliacao` : nota do produto.
- `char descricao[120]` : descricao curta.
- `struct NoAVL` : no da arvore AVL usada para buscar por codigo.
- `NoAVL* esquerda` e `NoAVL* direita` : ponteiros para os filhos da arvore.
- `struct IndiceNome` : guarda nome, codigo e posicao para busca por nome.
- `struct ResultadoDesempenho` : guarda dados do relatorio de desempenho.
- `const string ARQUIVO = "produtos.dat"` : nome fixo do arquivo de produtos.

3. Parametros e referencias

- `const string& texto` : recebe uma string sem copiar e sem permitir alteracao. Se o nome fosse `mensagem`, `const string& mensagem` faria a mesma coisa.
- `vector<Produto>& produtos` : recebe a lista original para poder alterar.
- `const vector<Produto>& produtos` : recebe a lista sem copiar e sem permitir alteracao.
- `Produto& produto` : recebe um produto original para editar.
- `const Produto& p` : recebe um produto sem copiar e sem alterar.
- `NoAVL* indiceAVL` : ponteiro para a raiz da arvore AVL.
- `NoAVL*& raiz` : referencia para o ponteiro da raiz, permitindo trocar a arvore.
- `int indiceAtual = -1` : parametro com valor padrao.

4. Entrada, saida e texto

- `cout <<` : mostra informacoes na tela.
- `cin >>` : le valores digitados, como numeros e caracteres.
- `getline(cin, texto)` : le uma linha inteira de texto.

- `cin.ignore(...)` : descarta caracteres que sobraram no buffer.
- `numeric_limits<streamsize>::max()` : quantidade maxima usada para limpar o buffer.
- `strncpy(destino, texto.c_str(), tamanho - 1)` : copia texto respeitando o limite do campo.
- `destino[tamanho - 1] = '\0'` : garante fim correto do texto em `char[]`.
- `transform(...)` : percorre o texto e aplica uma transformacao.
- `tolower(c)` : transforma caractere em minusculo.

5. Arquivo binario

- `ifstream arquivo(ARQUIVO, ios::binary)` : abre o arquivo para leitura binaria.
- `ofstream arquivo(ARQUIVO, ios::binary | ios::trunc)` : abre para gravacao e limpa o conteudo antigo.
- `arquivo.read(...)` : le dados do arquivo.
- `arquivo.write(...)` : grava dados no arquivo.
- `reinterpret_cast<char*>(&p)` : trata o produto como bytes para ler.
- `reinterpret_cast<const char*>(&p)` : trata o produto como bytes para gravar.
- `sizeof(Produto)` : tamanho, em bytes, de um produto.

6. Vector e produtos

- `produtos.push_back(p)` : adiciona produto ao final da lista.
- `produtos.empty()` : verifica se a lista esta vazia.
- `produtos.clear()` : remove todos os produtos da lista.
- `produtos.erase(produtos.begin() + pos)` : remove apenas o produto da posicao escolhida.
- `salvarProdutos(produtos)` : grava a lista atual no arquivo.
- `mostrarProduto(produtos[pos])` : mostra o produto encontrado.

7. Arvore AVL

- `altura(no)` : retorna a altura do no.
- `fatorBalanceamento(no)` : verifica a diferenca entre esquerda e direita.
- `rotacaoDireita(...)` : rebalanceia a arvore para a direita.
- `rotacaoEsquerda(...)` : rebalanceia a arvore para a esquerda.
- `inserirAVL(...)` : insere codigo e posicao no indice.
- `buscarIndiceAVL(...)` : procura o codigo e retorna a posicao no `vector`.
- `liberarAVL(...)` : libera a memoria da arvore.
- `construirIndiceAVL(produtos)` : monta a arvore com os produtos.
- `reconstruirIndiceAVL(...)` : recria a arvore apos cadastro, edicao ou remocao.

8. Cadastro, edicao e remocao

- `cadastrarProduto(...)` : le os dados e cadastra um novo produto.
- `listarProdutos(...)` : mostra todos os produtos.
- `editarInformacoes(...)` : abre o menu de edicao.
- `limparProdutos(...)` : remove todos os produtos apos confirmacao.
- `removerProduto(...)` : remove somente um produto pelo codigo.
- `confirmacao == 's' || confirmacao == 'S'` : aceita confirmacao em minusculo ou maiusculo.

9. Busca, filtros e ordenacao

- `buscarPorNome(...)` : busca produtos pelo inicio do nome.
- `construirIndiceNomeOrdenado(...)` : cria indice ordenado para busca por nome.
- `buscaBinariaPrimeiroNome(...)` : encontra a primeira posicao possivel da busca.

- `filtrarCategoria(...)` : mostra produtos da categoria digitada.
- `filtrarPreco(...)` : mostra produtos entre preco minimo e maximo.
- `quickSortPreco(...)` : ordena produtos por preco.
- `quickSortAvaliacao(...)` : ordena produtos por avaliacao.
- `sort(...)` : ordena usando funcao pronta da biblioteca.
- `swap(v[i], v[j])` : troca dois produtos de posicao.

10. Relatorio e menu

- `chrono::high_resolution_clock::now()` : pega o tempo atual para medir desempenho.
- `duration_cast<chrono::microseconds>(...).count()` : converte o tempo para microssegundos.
- `adicionarResultado(...)` : adiciona uma linha ao relatorio.
- `imprimirRelatorioDesempenho(...)` : mostra o relatorio comparativo.
- `void menu()` : controla o menu principal.
- `do { ... } while (opcao != 0)` : repete o menu ate sair.
- `switch (opcao)` : escolhe qual parte executar.
- `case 10` : chama `removerProduto`.
- `case 11` : chama `limparProdutos`.
- `break` : encerra o caso atual do `switch`.
- `int main()` : ponto inicial do programa.
- `return 0` : indica que terminou corretamente.